Lecture 10: TCP Client-Server Programming II

Objectives:

- Learn about the **Socket** class
- Learn how to write a TCP server using the **Socket** class
- Learn how to write a TCP client using the **Socket** class
- Learn how to Handle Text in Socket applications
- Learn about some problems that can occur in TCP and how to handle them.

1. The Socket class

Although many TCP/UDP client-server applications can be developed using the **TcpListener, TcpClient** (and **UdpClient)** classes, to really have full control on the behavior of such applications, one need to use the **Socket** class.

In particular, the **Socket** class must be used for applications involving protocols other than TCP/UDP and applications that require asynchronous communications.

This **Socket** class is in the **System.Net.Sockets** namespace. It is used to create a socket instance, which can then be used for network communication. The class has methods for creating both server and client sockets.

The constructor of the Socket class has the following format:

```
public Socket(AddressFamily af, SocketType st, ProtocolType pt)
```

Each parameter type in the above constructor is an enumeration type in the **System.Net.Sockets** namespace.

AddressFamily represent the addressing scheme being used for the communication.

Some of the values of this enumeration are: **InterNetwork**, **InterNetworkV6**, **DataLink**, **AppleTalk**, **Ipx**, etc.

InterNetwork represent IP v 4 addressing scheme, and our programs in this course will be centered around this address family.

SocketType represnts the type of socket being created, while **ProtocolType** indicates the type of protocol being used to transfer data on the socket.

In the **InterNetwork** address family, we have only the following combinations of SocketTypes and ProtocolTypes:

Socket Type	Protocol Type	Description
Dgram	Udp	Connection-less
Stream	Tcp	Connection-oriented
Raw	Icmp	Internet Control Message Protocol
Raw	Raw	Plain IP packets

The following statement shows how to create a Socket instance:

```
Socket server = new Socket(AddressFamily.InterNetwork,
      SocketType.Stream, ProtocolType.Tcp);
```

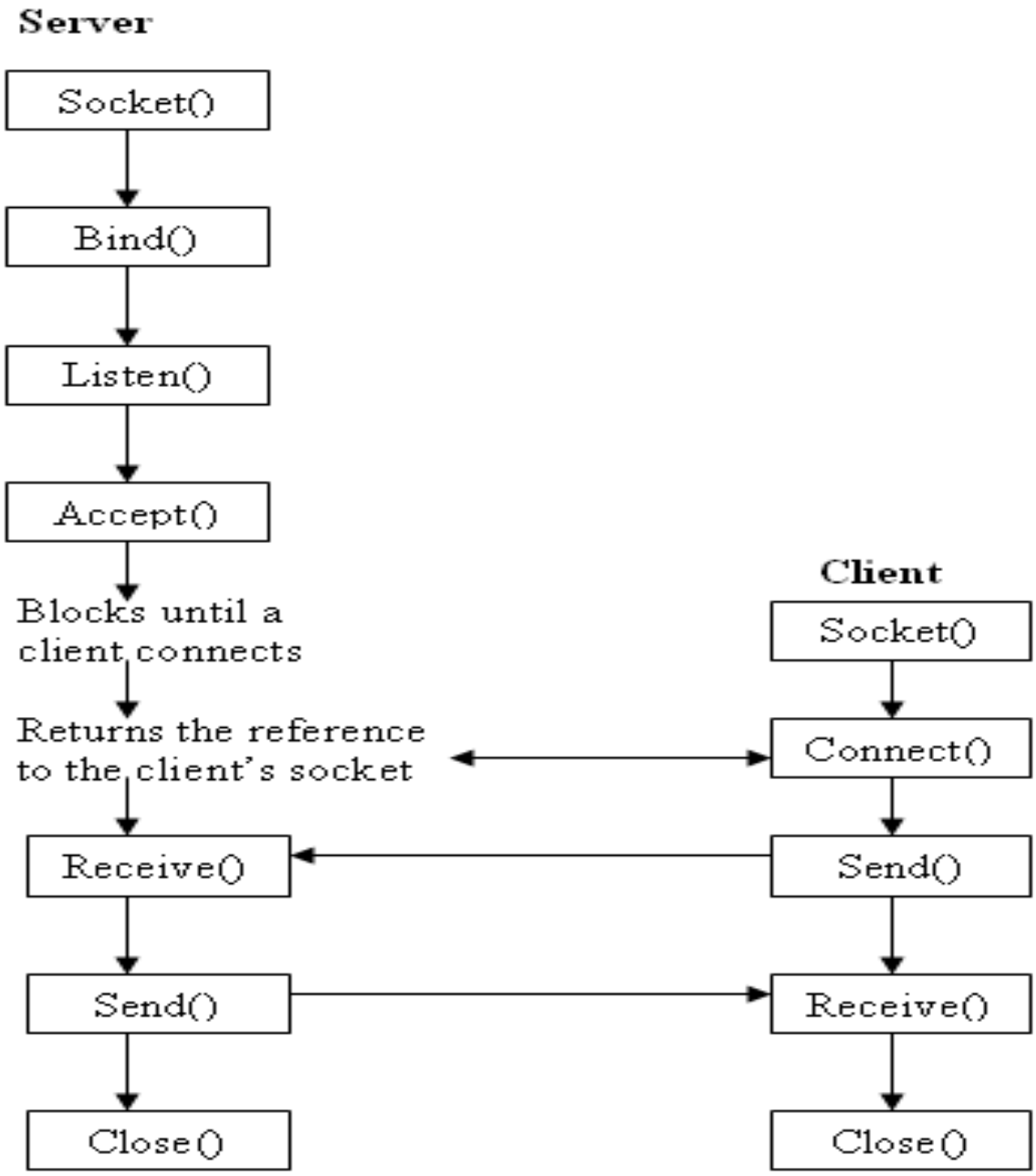
Methods:

The following are some of the methods of the Socket class. Some of these methods are used by Server sockets while others are for client sockets.

Method	Description	Applicable to
void Bind(IPEndPoint ep)	binds a server socket to a local end-point	Tcp Server
void Listen(int max)	listen for clients; max is the maximum number of clients to enqueue, while waiting for connection	Tcp Server
Socket Accept()	connects to a client and returns a reference to the client's socket	Tcp Server
void Connect(IPEndPoint)	connects to a remote end-point	Tcp Client

int Receive(byte[] data) int Receive(byte[] data, int size, SocketFlags sf) int Receive(byte[] data, int offset, int size, SocketFlags sf)	overloaded,: reads bytes from a socket	Tcp server/client
int Send(byte[]) int Send(byte[] data, int size, SocketFlags sf) int Send(byte[] data, int offset, int size, SocketFlags sf)	overloaded, sends bytes to a socket	Tcp server/client
void ReceiveFrom(byte[], ref EndPoint)	overloaded, receives from a client at EndPoint	Udp client
void SentTo(byte[] ref EndPoint)	overloaded, sends to a client at EndPoint	Udp client
void Shutdown(SocketShutdown how)	Disables sends and receives on a socket	All
void Close()	close a socket.	All

The following figure shows how some of these methods may be called by a Tcp client-server application.


Server
Socket()
Bind()
Listen()
Accept()
Blocks until a client connects
Returns the reference to the client's socket
Receive()
Send()
Close()
Client
Socket()
Connect()
Send()
Receive()
Close()

2. Creating a TCP Server Using the Socket class

The following example shows how to use the Socket class to create a server.

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

class SimpleTcpSocketServer {
  public static void Main() {
     Socket server = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
     IPEndPoint localEP = new IPEndPoint(IPAddress.Any, 9050);
     server.Bind(localEP);
     server.Listen(10);

     Console.WriteLine("Waiting for Client...");
     Socket client = server.Accept();
     IPAddress clientAddress =
((IPEndPoint)client.RemoteEndPoint).Address;
     Console.WriteLine("Got connection from "+clientAddress);

     byte[] data = Encoding.ASCII.GetBytes("Welcome to my test
server");
     client.Send(data);

     int dataSize = 0;
     while(true) {
        data = new byte[1024];
        dataSize = client.Receive(data);
        if (dataSize == 0)
          break;

        Console.WriteLine(Encoding.ASCII.GetString(data,0, dataSize));
        client.Send(data, dataSize, SocketFlags.None);
     }
     client.Close();
     server.Close();
  }
}
```

3. Creating a TCP Client Using the Socket class

The following example shows how to use the Socket class to create a client.

```
using System;
using System.Net;
using System.Net.Sockets;
using System.IO;
using System.Text;

class SimpleTcpSocketClient {
  public static void Main() {
     Socket socket = new Socket(AddressFamily.InterNetwork,
SocketType.Stream, ProtocolType.Tcp);
     IPEndPoint remoteEP = new
IPEndPoint(IPAddress.Parse("127.0.0.1"), 9050);
     try {
        socket.Connect(remoteEP);
     }catch (SocketException e) {
        Console.WriteLine("Unable to connect to server. ");
        Console.WriteLine(e);
        return;
     }

     byte[] data = new byte[1024];
     int dataSize = socket.Receive(data);
     Console.WriteLine(Encoding.ASCII.GetString(data, 0, dataSize));

     String input = null;
     while (true) {
        Console.Write("Enter Message for Server <Enter to Stop>: ");
        input = Console.ReadLine();
        if (input.Length == 0)
          break;
        socket.Send(Encoding.ASCII.GetBytes(input));
        data = new byte[1024];
        dataSize = socket.Receive(data);

        Console.WriteLine("Echo: "+ Encoding.ASCII.GetString(data, 0,
dataSize));
     }
     Console.WriteLine("Disconnecting from Server..");
     socket.Shutdown(SocketShutdown.Both);
     socket.Close();
  }
}
```

Notes:

You need to be aware that by default, the **Receive()** and **Send()** methods assume that the bytes array passed to them is full of data.

If some cells in the array are blanks, then you must specify the actual size of the data.

Also by default, these methods assume the data starts from index zero of the array. If this in not the case, then you must specify an **offset** value.

For example, in the **SimpleTcpSocketServer** class, two versions of the **Send()** methods were used.

In the first case, we used the version of the **Send()** method that takes only the bytes array.

This is because we are sure that the array returned by the GetBytes method in the preceding line has no empty cells.

However, in the second case, we have to use the version of **Send()** method that takes the size of the data.

This is because we are not sure whether the array returned by the preceding call to the Receive method is full of data or not.

In this case, we should use the version of Send that takes size of data and we should pass to it the actual size returned by the **Receive()** method.

Of course this value will only be correct if the **Send()** method is used properly in the client.

4. Handling Text in Socket applications

We saw that the **TcpClient** class has a method, **GetStream()**, that returns a **NetworkSream** instance, which can then be used to create **StreamReader** and **StreamWriter** instances. These instances simplify handling text files through their **ReadLine()** and **WriteLine**() methods.

Now does the **Socket** class has a method similar **GetStream()**? The answer is no. However, the **NetworkStream** class has a constructor that accepts a Socket instance as argument.

Thus, after accepting a connection from a client (in the case of server) or after connecting to a server (in the case of a client), the resulting socket can be used to create a **NetworkStream** instance, which can then be used to create **StreamReader** and **StreamWriter** instances as before.

The following example shows how to do this.

In the Server:

```
Socket client = server.Accept();
IPAddress clientAddress =
((IPEndPoint)client.RemoteEndPoint).Address;
Console.WriteLine("Got connection from "+clientAddress);

NetworkStream stream = new NetworkStream(client);
StreamReader reader = new StreamReader(stream);
StreamWriter writer = new StreamWriter(stream);
```

In the Client:

```
IPEndPoint remoteEP = new
IPEndPoint(IPAddress.Parse("127.0.0.1"), 9050);
try {
  socket.Connect(remoteEP);
}catch (SocketException e) {
  Console.WriteLine("Unable to connect to server. ");
  Console.WriteLine(e);
  return;
}

NetworkStream stream = new NetworkStream(socket);
StreamReader reader = new StreamReader(stream);
StreamWriter writer = new StreamWriter(stream);
```

5. Problems to be aware of in TCP Communication

Too Small Buffer Size:

In our simple example, a byte array of 1024 was used as the buffer size for the **Send** and **Receive** method calls.

This worked fine because the program was running in a controlled environment. That is both server and client know that the message size will not be more than this size.

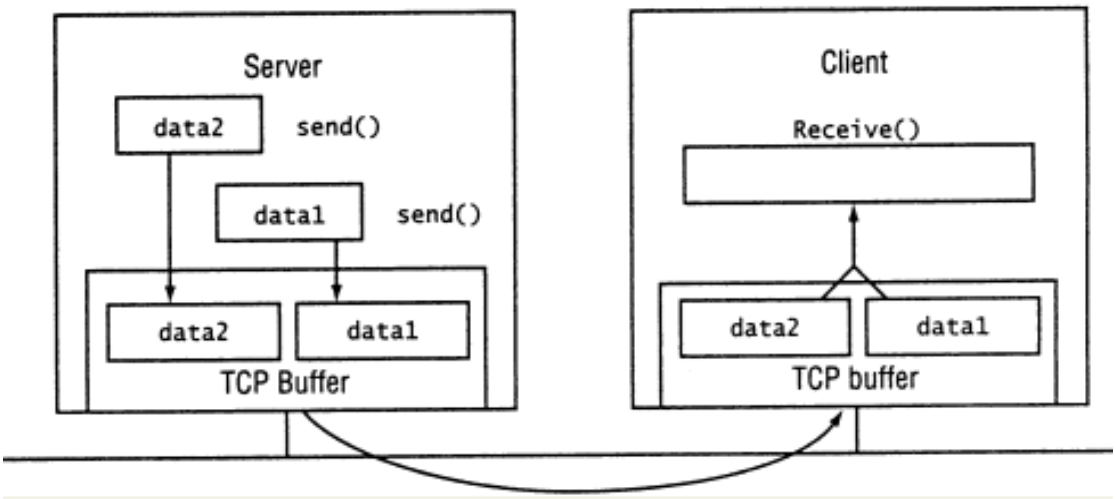
In real world however, you may not know the size of the data. So what happens if more data arrives than the buffer size?

Message Boundary Problem:

Another problem with TCP communication is that due to its connection-oriented nature, messages are considered to form a continuous stream of bytes.

This is implemented using TCP internal buffers, which are used to store messages until they are Received/Sent by applications.

The following figure shows this set-up.



The implication here is that TCP does not respect message boundaries. That is, there is no one-to-one correspondence between number/size of individual messages sent and the number/size of individual messages received.

Of course TCP pair will ensure that no data is lost. The problem is with applications. How will they know how many times they need to read before they collect the whole message?

Solutions:

1. For Text messages only, use ***ReadLine*** and ***WriteLine*** methods of the **StreamReader** and **StreamWriter** classes respectively.

   The advantage here is that the ***WriteLine*** will insert end-of-line markers in the message and the ***ReadLine*** will read one line at a time until there is no more lines to read.

   Notice here that the problem of too small buffer does not even arise in this case, since the ***ReadLine*** and ***WriteLine*** methods will take care of this.

2. Send the **size** of the message before sending the message. This is applicable for any type of data where ***Send*** and ***Receive*** methods are used for sending and Receiving.

   The idea here is, since the receiver knows the total size of the data, it will read the data in a loop, each time taking note of the actual size returned by the ***Receive*** method and updating the amount so far read until the entire size is read.

   A loop similar to the following is used.

```
int total = int.Parse(reader.ReadLine());
byte [] buffer = new byte[1024];
int recv = 0;
int sofar = 0;

while (sofar < total) {
  recv = s.Receive(buffer);
  process(buffer, recv);
  sofar += recv;
}
```